

CivicSuite — User Manual

****Last verified:**** 2026-05-28

This is the orientation manual for the CivicSuite umbrella repo. It is written in three parts plus a glossary:

1. ****For municipal decision-makers**** — non-technical overview. 2. ****For developers and IT staff**** — how the umbrella repo works. 3. ****Architecture reference**** — pointer to ARCHITECTURE.md.

For an at-a-glance honest module status, read STATUS.md first. For common operator questions, read FAQ.md.

Part 1 — For municipal decision-makers

What is CivicSuite?

CivicSuite is an ****open-source municipal product family****. It is not one giant program. It is a planned collection of modules a city can install one at a time, on its own hardware, on its own schedule. Cities do not need to send operational data to a vendor cloud, do not pay per seat, and can inspect or modify the source code.

Current honest state

CivicSuite is under release-recovery review. Public "shipping," "product-ready," and "v1.0.0 proves release maturity" claims are frozen until each repo re-earns that status through the gates in docs/release-recovery-status.md.

- `civiccore` is the shared platform; v1.2.0 is the current city-core platform release. - `civicrecords-ai` (FOIA / records) is the current developer-preview records release car at v1.7.3. - `civicclerk` (meetings) is the current meeting workflow release car at v1.0.3. - `civiccode` is the current municipal-code release car at v1.0.8. - CivicAccess is out of city-core pending gap closure and re-probe. CivicZone, CivicPlan, CivicPermit, and CivicInspect are queued Tier 2 modules, not city-core products. - The city-core installer has predecessor Linux and Windows matching-host lifecycle, integration, first-run browser QA, documentation lockstep, and CI/PR evidence. The current Stage 2 live-installer branch is under fix-now audit-team closure for installer, LLM-proof, docs, and clean-VM altitude findings. It is not public-use ready, procurement-ready, production-ready, macOS lifecycle certified, or a release of the full suite. - The suite launcher is the local browser front door for staff, resident, and IT-admin orientation over the installed services. The 2026-05-28 engagement is proving the shared browser session across modules; until the independent audit-team rerun clears it, treat that as in-flight city-core beta work rather than a municipal managed-SSO claim.

A municipality cannot today run end-to-end on this suite. Pilot evaluation is reasonable; procurement is not.

What this means for your city

- **No vendor lock-in.** You run the software on your own hardware. - **No mandatory cloud dependency.** Local-first by design. - **No per-seat billing.** The Apache 2.0 license forecloses metering. - **You can evaluate one module at a time.** No need to adopt the whole suite. - **You still own operations.** Not a managed SaaS. Your IT team, or a contractor you choose, runs install / upgrades / recovery.

Part 1.5 - Your first task with the city-core launcher

If you want to *try* CivicSuite today, start at the suite launcher. Treat this as a developer-preview walkthrough, not production setup.

Prerequisites

- A workstation with **8+ CPU cores**, **32 GB RAM**, **50 GB free disk space**. The local AI response-letter proof uses `gemma4:e4b`; Docker Desktop / WSL2 must expose enough memory for Ollama to load that model. The installer readiness floor is 12 GB RAM, but 32 GB remains the practical evaluation target. - **Docker Engine** on Linux, or Docker Desktop on Windows/macOS for wrapper-based installs. Linux is the primary development and runtime proof path. On Windows, also WSL 2 + Virtual Machine Platform. The city-core first-run wizard offers a Guided Setup path for supported Linux and Windows hosts, plus a Manual Prerequisite path for IT-managed machines. Linux Guided Setup uses Docker's signed package repositories where supported. macOS remains beta/archive/readiness only. - About 30 minutes for first install (model downloads).

Install (Windows wrapper)

1. Download the current city-core installer artifact from the active run evidence or the matching GitHub release attestation when one is published. 2. Verify the SHA-256 checksum or release manifest from the same source. 3. Run the installer. **Windows SmartScreen** can warn "Unknown publisher" - this is expected for unsigned beta artifacts. Click "More info" only when the checksum matches the trusted artifact source. 4. Choose Guided Setup if Docker Desktop/WSL2 is missing on Windows or Docker Engine is missing on Linux. Choose Manual Prerequisite if IT already installed those components. 5. Complete the first-run wizard, rotate the first administrator password, then open the suite launcher at the URL printed by the installer. 6. From the launcher, open Records AI, CivicClerk, or CivicCode. In the current engagement, shared-session behavior is being verified across those three staff modules.

Install (Linux / macOS)

Linux is the primary runtime target. Windows is supported through a wrapper around the same containerized services. macOS remains beta/archive/readiness only until matching-host lifecycle evidence exists.

```
git clone https://github.com/CivicSuite/civicrecords-ai.git
cd civicrecords-ai
bash install.sh
```

For city-core, the first-run wizard can guide supported Linux/Windows prerequisite setup, then resumes the install. For IT-managed machines, choose Manual Prerequisite after Docker/WSL is already present. macOS prerequisite bootstrap remains out of scope for this run.

If the installer fails

The installer writes a machine-readable lifecycle report and streamed logs under ``installer/reports/<run-id>/``.

- Full lifecycle JSON: ``installer/reports/<run-id>/clerk-core-installer-lifecycle.json`` - Streamed package logs: ``installer/reports/<run-id>/launcher-output/*.log`` - Suite launcher port: ``18082``

If the suite launcher check fails, free port ``18082`` before retrying. On Windows, run ``netstat -ano | findstr :18082``, identify the owning PID, and stop that process from Task Manager or an elevated PowerShell prompt. If Ollama reports a model-load failure, increase Docker Desktop / WSL2 memory or choose a smaller supported model only after the release manifest and docs name that model. A slow Ollama prewarm is a warning; an actual model-load failure blocks the install because the Records AI response-letter proof must be model-generated, not a template fallback.

``--verify-existing-install-root`` only verifies stacks with a matching ``civicsuite-install-provenance.json``. If that provenance file is missing or its manifest hash/source commits do not match the current package, reinstall or repair from the current package before rerunning verification.

Trust path for city-core artifacts

City-core artifacts for this run are live regenerated evidence artifacts, not restored committed ``installer/dist`` files. Before testing a package:

1. Use the active run evidence path recorded in README.md and STATUS.md.
2. Verify the generated ``SHA256SUMS`` or release-manifest hash for the package you will run.
3. Confirm the source pins in ``installer/modules.json`` match the vendored source commits copied into the package.
4. For CivicCode release-car assets, confirm the published SHA256 and attestation assets recorded in the module release evidence.

Do not restore old generated installer artifacts unless Scott explicitly decides that those artifacts should be restored.

First task: search a small document set

1. Open `<http://localhost:8080>` in your browser.
2. Sign in with the initial administrator credential file surfaced by the installer, rotate it immediately, and store the rotated value in your municipal password vault.
3. **Sources** → **Add Source** → enter a directory path with a few sample PDFs/DOCX files.
4. Click **Ingest Now**. The pipeline parses, chunks, embeds, and indexes the documents.
5. **Search** → type a natural-language query (e.g., "what does the city pay for streetlight maintenance?"). Results show with source attribution and relevance scores.

What you should expect

- **What works.** Document ingestion, hybrid search with citations, exemption flagging (with required human review), request lifecycle from intake to closure, fee tracking, response letter drafting. - **What's a developer preview.** SMTP delivery into status transitions is pending; full third-party accessibility audit is pending; production deployments are unverified. - **What's not in scope.** SharePoint and IMAP connectors are roadmap; MIT-style "drop a CSV in this folder" is the most reliable connector path today.

Where to go next

- For real evaluation: stand up a non-production tenant, ingest a representative document corpus, and run a week of internal staff requests through it. - For procurement: wait for a later procurement-readiness gate; this beta-ready truth-reconciled package is not procurement-ready. - For development: read CONTRIBUTING.md and the records-ai `docs/`.

Part 2 — For developers and IT staff

What lives in the umbrella repo?

The `civicsuite` repo is **documentation-, governance-, and coordination-first**. It contains:

- the [Charter](CHARTER.md), [Continuity plan](SUCCESSION.md), [Compatibility matrix](docs/compatibility/index.md), [Roadmap](docs/roadmap/index.md) - the [Shared extraction consumer rollout playbook](docs/roadmap/shared-extraction-consumer-rollout.md) - the unified suite specification and module catalog under `docs/` and `specs/` - suite-level governance and ADRs - the **suite installer** under `installer/` (a prior Clerk-Core starter release path plus the active city-core profile work) - verification scripts under `scripts/`

The umbrella does **not** contain runtime code for individual products — that lives in the per-module repos.

Module repos

| Repo | Status (see STATUS.md for full detail) | |---|---| | `civicrecords-ai` | v1.7.3 developer-preview city-core records release car | | `civiccore` | Shared platform; v1.2.0 current city-core platform | | `civicclerk` | v1.0.3 meeting workflow city-core release car | | `civiccode` | v1.0.8 municipal-code city-core release car | | `civicaccess` | OUT of city-core after NEEDS-WORK depth probe | | `civiczone`, `civicplan`, `civicpermit`, `civicspect` | Queued Tier 2 modules on demotion-truth labels | | `civicgrants`, `civicprocure` | v0.2.0 scaffold-depth recovery labels | | All others | Foundation surfaces (v0.1.x) | | `civicregwatch`, `civicapi` | Planned; spec only |

Dependency rule

Modules depend on `civiccore`. `civiccore` never depends on modules.

How to evaluate a module

1. Read the module's ``README.md``. 2. Read the module's ``USER-MANUAL.md``. 3. Read the module's ``CHANGELOG.md``. 4. Follow the module's ``CONTRIBUTING.md`` install steps on a clean machine. 5. Run the module's verification gates (``scripts/verify-release.sh`` if present). 6. Cross-check against `[STATUS.md](STATUS.md)` and `[docs/release-recovery-status.md](docs/release-recovery-status.md)` before promoting any claim about the module.

How releases are coordinated

When shared-platform behavior changes:

1. ``civiccore`` ships first. 2. Consumer modules adopt via a bounded rollout. 3. The compatibility matrix is updated. 4. Current-facing docs are updated in both the consumer repo and the umbrella repo when suite-level status changes.

The current standardized consumer adoption process: `[docs/roadmap/shared-extraction-consumer-rollout.md](docs/roadmap/shared-extraction-consumer-rollout.md)`.

How to contribute

- Suite-wide roadmap, governance, compatibility, or umbrella documentation work belongs in this repo. - Product / module bugs and features belong in the relevant module repo. - The umbrella repo also holds the suite-installer scaffolding and verification scripts; contributions to those land here.

Start with `[CONTRIBUTING.md](CONTRIBUTING.md)` and `[docs/governance/index.md](docs/governance/index.md)`.

Part 3 — Architecture reference

The full suite architecture, dependency graph, data-flow rules, and CivicCore extraction phasing live in `[ARCHITECTURE.md](ARCHITECTURE.md)`. The text-and-ASCII summary below is a quick orientation; the full diagram is in `ARCHITECTURE.md`.

Suite topology (generated summary)

<!-- BEGIN GENERATED SUITE TOPOLOGY -->

Generated from ``installer/modules.json``. Re-run ``python scripts/docs/render_topology.py --check`` before publishing docs.

- Root installer truth label: ``city_core_beta_ready_truth_reconciled``. - City-core profile status: ``beta_ready_truth_reconciled``. - City-core modules are the only modules represented in the current beta-ready profile. - Disabled profiles and excluded modules remain documented as out of scope until their own gates clear.

Module	Version	Role	Dependencies	Source commit	Installer status	
CivicCore	1.2.0	shared platform	none	`9f7e3a5a0156`	`v1_2_0_shared_ingestion_shipped`	
CivicRecords AI	1.7.3	records workflow	`civiccore`	`cddc4d2be856`	`v1_7_3_city_core_release_car`	
CivicClerk	1.0.3	meetings workflow	`civiccore`	`f39d0eeccc68`	`v1_0_3_city_core_release_car`	
CivicCode	1.0.8	municipal code	`civiccore`, `civicclerk`	`a960bba0a224`	`v1_0_8_city_core_release_car`	

Excluded from city-core: - `civicaccess`: OUT after NEEDS-WORK depth probe on branch probe/civicaccess-depth-2026-05-23; re-evaluate only after gap closure and re-probe.

Disabled profiles: - `land-use` (Land Use): queued - depends on Tier 2 module work after city-core ships - `full-suite` (Full Suite): 15 modules remain pre-1.0 scaffolds pinned to older CivicCore lines; not installable as a coherent suite

<!-- END GENERATED SUITE TOPOLOGY -->

Upgrade and migration order

When `civiccore` ships a backward-compatible change: 1. `civiccore` releases the new version. 2. Compatibility matrix is updated. 3. Consumers adopt via the rollout playbook.

When `civiccore` ships a breaking change: 1. Change is documented in advance (ADR). 2. `civiccore` releases first. 3. Consumers ship paired changes. 4. Compatibility matrix records the new pairing.

Continuity and governance

Continuity is now a gate, not a future aspiration. See SUCCESSION.md. The roadmap that governs the rest of the program is docs/roadmap/index.md.

Glossary

- **ADA Title II** — accessibility compliance requirement for public-sector digital services. Cities >50K must comply by 2027; smaller cities by 2028. - **ADR** — Architecture Decision Record. - **Air-gap** — operating without an outbound network connection. - **CC BY 4.0** — Creative Commons Attribution license used for documentation. - **CI** — continuous integration. - **Connector** — module-side adapter that imports data from a city system. Read-first by design. - **FOIA** — Freedom of Information Act and related public-records laws. - **LLM** — large language model. - **Local-first** — software designed to run on infrastructure the city controls. - **Module** — a single CivicSuite product (e.g., civicclerk, civicrecords-ai). - **Open source** — software available under a license allowing use, modification, and redistribution. - **Pinned version** — a specific exact dependency pairing rather than a range. - **Procurement-ready** — has passed the recovery gates and can be evaluated against city procurement standards. Distinct from "has a release tag." - **Provisional** — has a version label but the label is not currently a promotion claim. - **Recovery gate** — one of the ten checks documented in `docs/release-recovery-status.md` that must pass before a "shipping" claim can be re-issued. - **Sovereign deployment** — software running on infrastructure the city controls. - **Sigstore / cosign** — signing tooling used for `civiccore` release attestations starting v0.22.1. - **WCAG 2.2 AA** — Web Content Accessibility Guidelines target for public-facing surfaces. - **Wheel** — Python package distribution format.

When something goes wrong

| Symptom | Where to look | |---|---| | Module will not install | The module repo's `README.md` and `CONTRIBUTING.md`; if you hit a Docker/WSL prereq, choose Guided Setup on supported Linux/Windows hosts or Manual Prerequisite for IT-managed environments. Linux Guided Setup uses Docker's signed package repositories where supported. | | Suite launcher opens but shows no module activity | Run the installer verify command, confirm Docker is running, then refresh the launcher. The launcher session is local browser/runtime state; it is not a municipal SSO proof. | | Unsure whether an artifact is current | Verify the live `SHA256SUMS` or release manifest from the active run evidence path; do not rely on restored `installer/dist` files unless Scott explicitly approved restoration. | | `civcore` version mismatch | docs/compatibility/index.md is the canonical pairing source. | | README says "shipping," recovery doc says "frozen" | The recovery doc wins. See docs/release-recovery-status.md. | | Unsure where to file a bug | CONTRIBUTING.md — bug-routing decision tree. | | Security issue | SECURITY.md — open a private GitHub Security Advisory. | | General support question | SUPPORT.md. | | First-time evaluator | FAQ.md, then STATUS.md, then this manual. |